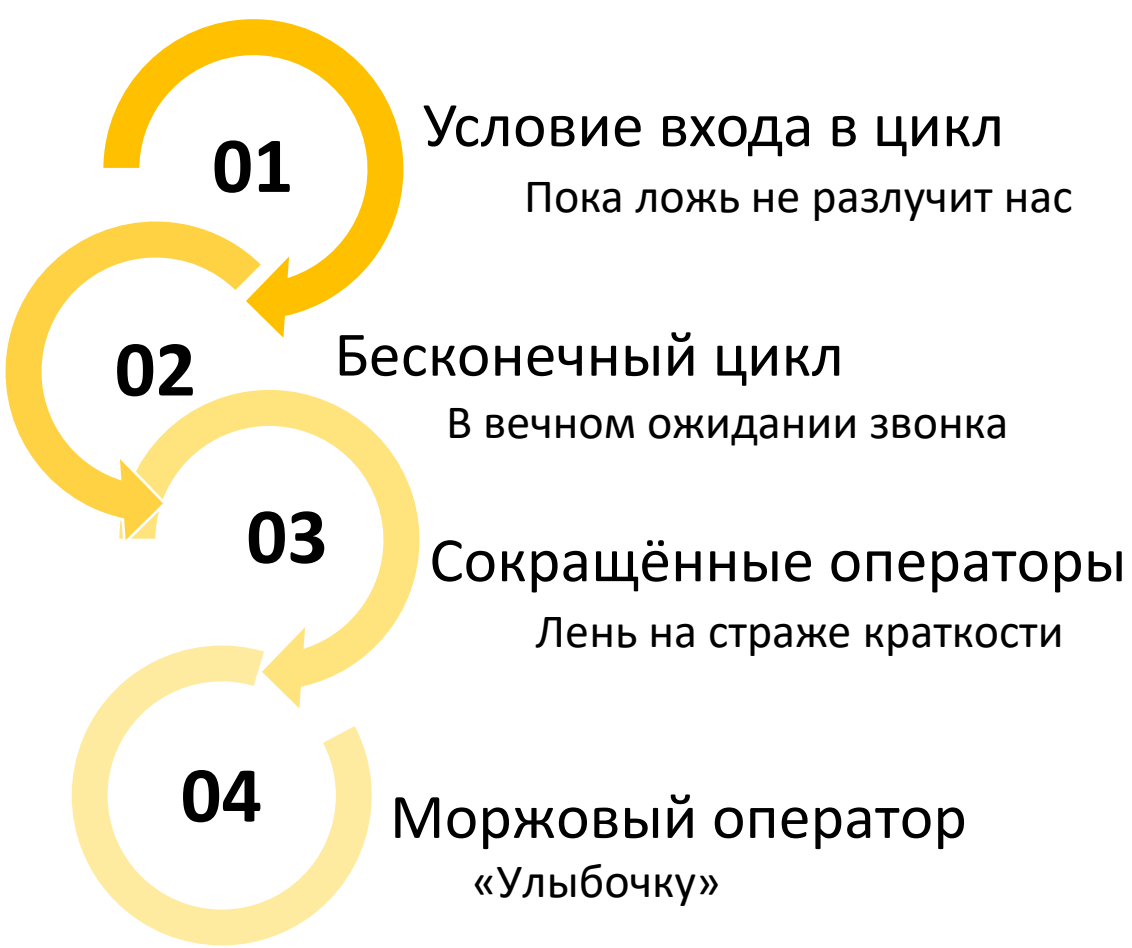

Яндекс

Яндекс Лицей

Цикл while

Учимся крутиться как белка в колесе

Что впереди

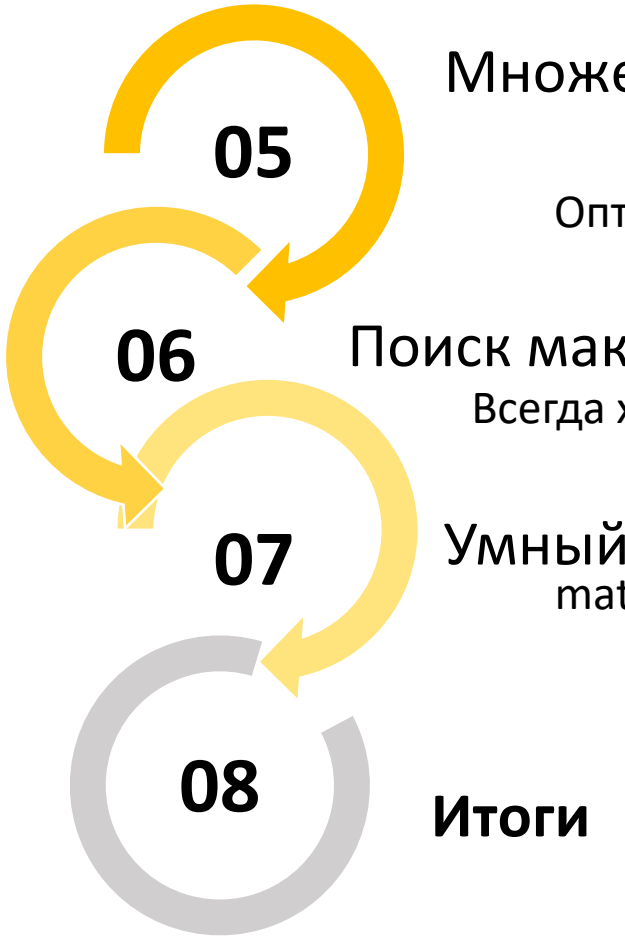


01 Условие входа в цикл
Пока ложь не разлучит нас

02 Бесконечный цикл
В вечном ожидании звонка

03 Сокращённые операторы
Лень на страже краткости

04 Моржовый оператор
«Улыбочку»



05 Множественное присваивание
Оптом дешевле

06 Поиск максимума
Всегда хочется большего

07 Умный выбор
match/case внутри

08 **Итоги**



Цикл while

Учимся крутиться как белка в колесе

Пока ложь не разлучит нас



Цикл **while** («пока») называют циклом с неизвестным количеством повторений. Блок кода внутри while выполняется до тех пор, пока истинно условие в заголовке цикла.

Запись while очень похожа на запись if:

```
while <условие>:  # Заголовок.
    <блок кода>  # Тело цикла. Не забываем про 4 пробела.
```

Один оборот цикла (выполнение всего тела цикла) называют **итерацией**.

Условие цикла записывается как и для if:

- с помощью операций отношения (>, >=, <, <=, !=, ==);
- с использованием логических операций not, and, or;
- с помощью переменных типа bool (или неявного приведения переменных других типов к bool).

Пока ложь не разлучит нас



А это мы проверяем появление результатов важного экзамена каждую минуту, пока они не появились:

```
got_results = False  # Выполнится перед началом цикла.

while not got_results:  # not от False -> True,
                        пока нет результатов — нервничаем.
    print(" 🔄 Проверяю почту...")
    print("Результаты пришли?")
    if input() == 'да':
        got_results = True  # Ура!

print(" 📩 Письмо получено!")
# Выполнится после завершения цикла.
```

Посмотрим на программу, которая иллюстрирует длительную поездку с младшим братом. В ней цикл будет выполняться до тех пор, пока пользователь не введёт «да» или «приехали»:

```
question = 'Мы уже приехали?'
print(question)
answer = input()
while answer != 'да’ and answer != 'приехали':
    print(question)
    answer = input()
print('Отличная поездка!')
```

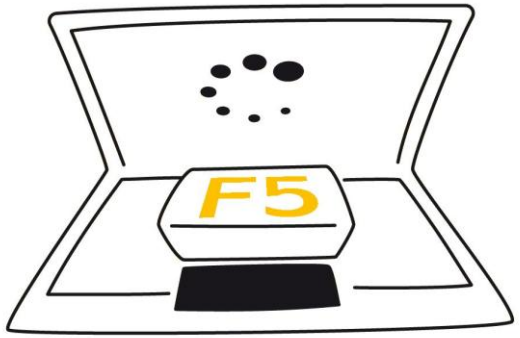
Если условие изначально ложно, то цикл не выполнится вообще ни разу!

Бесконечный цикл: в вечном ожидании звонка

Если условие изначально ложно, то цикл не выполнится ни разу, но что, если оно будет всегда истинно? Добро пожаловать в бесконечный цикл.

```
while True:
    print("∞ Бесконечность — не предел!")
```

Всегда проверяйте, есть ли в цикле условие для выхода!



Если вы случайно или специально сделали бесконечный цикл, то не надо перезагружать компьютер, можно его остановить, если в консоли набрать **Ctrl + C**.



Сокращённые операторы

Лень на страже краткости (или наоборот)

Сокращённые операторы



В операторе присваивания одно и то же имя переменной может стоять и справа (в составе какого-то выражения), и слева. В этом случае *сначала вычисляется правая часть* со старым значением переменной, после чего результат становится новым значением этой переменной.

```
counter = 0  # Было ноль.
counter = counter + 1  # А стало на один больше.
```

Это не уравнение, у которого надо привести подобные 😅

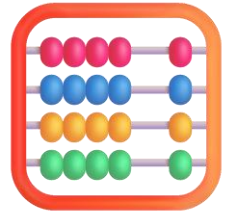
Встречайте: сокращённая форма записи оператора присваивания (для примера выше):

```
counter = 0  # Было ноль.
counter += 1  # А стало на один больше, да и ещё как красиво написано!
```

Такая форма записи есть у любого оператора:

+= (добавить)
-= (вычесть)
*= (умножить)
/= (разделить)
//= (целочисленно разделить)
**= (возвести в степень)
%= (найти остаток от деления)

Сейчас всё посчитаем



Очень часто такую сокращённую форму используют в циклах while для изменения переменной, которая участвует в проверке условия.

```
cookie_count = 10
while cookie_count > 0:
    print("Съел печеньку. Ом-ном-ном.")
    cookie_count -= 1  # ← Сокращённый оператор как диета для кода.
print("Теперь я без печенек.")
```

Не забываем две очень важные вещи:

- задать начальное значение переменной;
- изменять её в цикле, чтобы когда-нибудь из него выйти.

Сигнал остановки — наш спасительный стоп-кран



Рассмотрим такую задачу: пользователь вводит какие-нибудь значения, причем мы заранее не знаем, сколько их будет. Пусть это будут числа — возраст котиков. И пользователь решил посчитать суммарный возраст всех своих домашних животных. В этом случае надо подумать, как пользователь может нам сообщить о том, что ввод закончился.

Для сигнала остановки выбирают значение, которое точно не может встретиться в валидном пользовательском вводе.

Так как мы считаем возраст, условимся, что отрицательным он быть не может, поэтому ввод «−1» — сигнал остановки.

Называйте переменные осмысленно, пожалуйста, пожалуйста!

Если бы мы знали точно, что пользователю надо сосчитать суммарный возраст ровно трёх котиков, то можно было бы написать что-то вот такое и даже обойтись без цикла:

```
total_age = 0
current_cat_age = int(input())
total_age += current_cat_age
current_cat_age = int(input())
total_age += current_cat_age
current_cat_age = int(input())
total_age += current_cat_age
print('Суммарный возраст котиков', total_age)
```

Сигнал остановки — наш спасительный стоп-кран



Если вы написали какой-то кусочек кода дважды (или, о ужас, трижды), значит, вы что-то делаете не так.

Очевидно, что можно организовать цикл со счётчиком, который внутри цикла будет считать каждую итерацию. А условием выхода обозначим выполнение нужного количества итераций:

```
count = 0
total_age = 0
while count < 3:
    current_cat_age = int(input())
    total_age += current_cat_age
    count = count + 1
print('Суммарный возраст котиков:', total_age)
```



Однако у нас в задаче количество котиков неизвестно, поэтому понадобится цикл до ввода сигнала остановки (−1):

```
total_age = 0
print('Посчитаем суммарный возраст всех ваших котиков.')
print('Вводите возраст; для остановки введите -1.')
current_cat_age = int(input())
while current_cat_age != -1:
    total_age += current_cat_age
    current_cat_age = int(input())
print('Суммарный возраст котиков:', total_age)
```

Моржовый оператор (:=): «Улыбочку»



Оператор := (похож на моржовые клыки) позволяет присвоить значение переменной прямо в условии. Это упрощает код, объединяя присваивание и проверку.

Было:

```
print("Введи 'quit' для выхода")
text = input()
while text != "quit":
    print(f"Ты написал: {text}")
    text = input()
print("Выход!")
```

Стало:

```
print("Введи 'quit' для выхода")
while (text := input()) != "quit":
    print(f"Ты написал: {text}")
print("Выход!")
```

Скобки нужны для того, чтобы повысить приоритет присваивания (а он самый маленький). Благодаря скобкам сначала выполнится input() и результат пользовательского ввода запишется в переменную text, и только потом пользовательский ввод сравнится со строкой «quit».

Без скобок приоритета пользовательский ввод сначала сравнился бы со строкой, а потом результат этого сравнения (True или False) записался бы в переменную text.

Моржовый оператор (:=): «Улыбочку»



Ещё пример:

```
# Без моржа:
print("Введите пароль:")
text = input()
while len(text) < 8:
    print("Пароль слишком короткий!")
    print("Введите пароль:")
    text = input()
```

```
# С моржом (и небольшим жульничеством с параметром функции input()):
while len(text := input("Введите пароль:\n")) < 8:
    print("Пароль слишком короткий!")
```

Множественное присваивание. Оптом дешевле ⇄

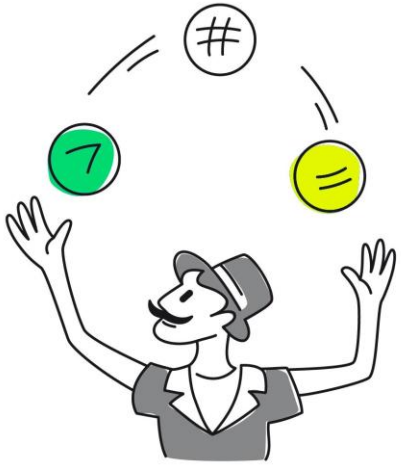
Если нам захочется поменять местами значения двух переменных, то естественное желание — сделать такой обмен через третью переменную:

```
right_hand = 'apple'
left_hand = 'orange'
# Для обмена подкинем что-то в воздух,
air = right_hand
# переложим,
right_hand = left_hand
# поймаем:
left_hand = air
```

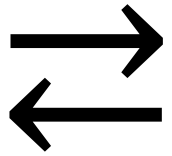
В Python благодаря магии *множественного присваивания* мы можем обойтись и без третьей переменной (Что? Да!):

```
right_hand = 'apple'
left_hand = 'orange'

right_hand, left_hand = left_hand, right_hand
# Переменные обменялись значениями.
```



Множественное присваивание



А ещё такой прием можно использовать, чтобы присвоить значения нескольким переменным в одной строке:



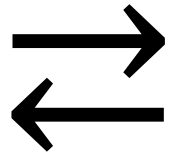
```
cat_name, dog_name = "Барсик", "Шарик"
```

Можно жонглировать любым количеством переменных:

```
# Обмен значениями трёх переменных
за один присест:
x, y, z = "огонь", "вода", "земля"
x, y, z = z, x, y
# Теперь x = "земля", y = "огонь", z = "вода".
```

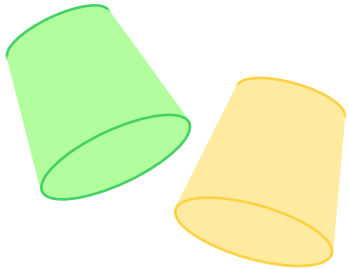
Не забываем пробелы после запятой, чтобы не прогневать **PEP8**.

Множественное присваивание



С этими знаниями давайте напишем программу, где мы будем каждый раунд перепрятывать «шарик» из одной переменной в другую.

```
print("Сейчас шарик в правом стаканчике!")
left_cup, right_cup = "", "шарик"
# Перед каждым запуском тихонечко, чтобы никто не видел,
меняем значения переменных.
swap_count = 9  # Ох, тут бы нам пригодилась возможность
генерировать случайное число!
current_round = 0
print("Кручу-верчу, запутать хочу")
while current_round < swap_count:
    left_cup, right_cup = right_cup, left_cup
    current_round += 1
print("В каком стаканчике шарик? 1 - в левом, 2 - в правом")
answer = int(input())
if answer == 1 and left_cup == "шарик" or answer == 2 and right_cup == "шарик":
    print("Вы победили!")
else:
    print("Получится в другой раз")
```





Поиск максимума.

Всегда хочется большего

Поиск максимума. Или минимума

В теле цикла каждый подходящий элемент данных обрабатывается операторами по принципу:

- если текущий элемент больше максимума, меняем максимум;
- если текущий элемент меньше минимума, заменяем минимум.

```
biggest_book = 0  # Любая книга длиннее.
smallest_book = float("+inf")  # Любая книга короче бесконечности.
while (n := int(input())) > 0:  # Сразу вводим, записываем в переменную и сравниваем.
    # Сравниваем с помощью if.
    if n > biggest_book:
        biggest_book = n
    # Или выбираем нужное значение с помощью min|max.
    smallest_book = min(smallest_book, n)
# Сюда попадём, когда введём 0 или отрицательное число.
print(biggest_book)
print(smallest_book)
```



Умный выбор:

match/case внутри

Умный выбор

В Python в любом блочном операторе может быть другой блочный оператор. Поэтому никто не запретит вам скомбинировать while и match/case, особенно это может пригодиться для создания каких-нибудь меню или игр с выбором варианта ответа, например лабиринтов.

```
health = 100
run_away = False

while health > 0 and not run_away:  # Пока есть здоровье и не убежали.
    print("Ваш ход (атака/защита/бег):")
    match input():  # Нам даже не надо хранить ввод — сразу выбираем.
        case "атака":
            print("⚔️ Атакую врага!")
            health -= 10  # Тратим силы.
        case "защита":
            print("🛡️ Защищаюсь!")
            health += 5  # Восстанавливаемся.
        case "бег":
            print("🏃 Бегу с поля боя!")
            run_away = True
        case _:
            print("🤷 Неизвестная команда.")

    print(f"❤️ Здоровье: {health}")

if health <= 0:
    print("💀 Игра окончена.")
else:
    print("🏆 Победа!")
```

Итог: цикл жизни нам стал чуть понятнее



Вы узнали:

Цикл while — это вечный двигатель вашей программы. Мы запустили этот двигатель, а также:

- начали экономить символы с сокращенной записью присваивания;
- научились жонглировать значениями переменных;
- смогли создать бесконечный цикл и прервать бесконечность с помощью Ctrl+C;
- познакомились с моржовым оператором без похода в зоопарк;
- а ещё рассмотрели много типовых примеров использования цикла while.

P. S. Даже звёзды когда-нибудь погаснут, а while True будет с вами всегда.